You said:

Explain Pandas in Python

ChatGPT said:

Pandas in Python – Complete Beginner-Friendly Explanation

1. What is Pandas in Python?

Pandas is a Python library used to store, organize, analyze, and manipulate data.

It makes working with data in tables much easier, especially when you need to perform operations such as filtering, sorting, updating, deleting, and analyzing data.

For example, suppose you have student records:

Roll No	Name	Marks
101	Rahul	85
102	Priya	90
103	Amit	75

Using Pandas, you can easily:

- Display all student records.
- Find students who scored more than 80 marks.
- Add new students.
- Update students' marks.
- Remove unwanted records.
- Calculate the average marks.



Think of Pandas as a powerful tool for working with table-based data in Python.

2. Why do we use Pandas?

Imagine you have an Excel file containing 10,000 employee records. You want to find all employees whose salary is greater than ₹50,000.

Without Pandas, writing code to read and process the file manually can be complicated.

With Pandas, you can do it in just a few lines.

```
import pandas as pd

data = {
    "Name": ["Rahul", "Priya", "Amit"],
    "Salary": [40000, 60000, 75000]
}

df = pd.DataFrame(data)

print(df[df["Salary"] > 50000])
```

Output:

```
    Name  Salary
1  Priya   60000
2   Amit   75000
```

Pandas is commonly used in:

- Data analysis and data science
- Machine learning and AI
- Reading Excel and CSV files
- Business reporting
- Financial data analysis
- Cleaning missing or incorrect data

3. How to install Pandas

Before using Pandas, you need to install it.

Open your terminal or PowerShell and execute:

```
pip install pandas
```

If you use Python through the Python launcher on Windows, you can also run:

```
py -m pip install pandas
```

To check whether Pandas is installed:

```
import pandas as pd

print(pd.__version__)
```

This displays the installed Pandas version.

4. How to import Pandas

After installing Pandas, import it into your Python program.

```
import pandas as pd
```

Here:

- `import` loads the library.
- `pandas` is the library name.

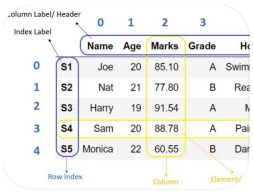
- `as pd` creates a short alias for Pandas.

Instead of writing `pandas.DataFrame()`, you can write `pd.DataFrame()`.

Important: `pd` is a conventional alias. You could choose another alias, but `pd` is the standard used in most examples and projects.

5. Two important Pandas data structures

Pandas provides two main data structures.



1. Series

A **Series** is a one-dimensional labeled collection of values.

Example:

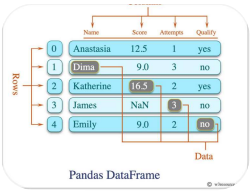
```
import pandas as pd

marks = pd.Series([85, 90, 75])
print(marks)
```

Output:

```
0    85
1    90
2    75
dtype: int64
```

Each value has an index, starting at `0` by default.



2. DataFrame

A **DataFrame** is a two-dimensional table containing rows and columns.

Example:

```
data = {
    "Name": ["Rahul", "Priya"],
    "Marks": [85, 90]
}
```

```
df = pd.DataFrame(data)
print(df)
```

Output:

```
    Name  Marks
0  Rahul     85
1  Priya     90
```

A DataFrame is similar to a table in Excel or a database.

The easiest way to remember the difference:

- **Series:** One-dimensional data.
- **DataFrame:** Two-dimensional tabular data.

6. Creating a DataFrame

Let's create a student table from scratch.

```
import pandas as pd

data = {
    "RollNo": [101, 102, 103, 104],
    "Name": ["Rahul", "Priya", "Amit", "Sneha"],
    "Marks": [85, 90, 75, 95]
}

df = pd.DataFrame(data)

print(df)
```

Output:

```
   RollNo   Name  Marks
0     101  Rahul     85
1     102  Priya     90
2     103   Amit     75
3     104  Sneha     95
```

Notice that Pandas automatically creates an index: `0`, `1`, `2`, and `3`.

You can also specify your own index:

```
df = pd.DataFrame(data, index=["A", "B", "C", "D"])

print(df)
```

Now the rows have labels `A`, `B`, `C`, and `D`.

7. Important Pandas operations

Let's use the following student DataFrame for the examples below.

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["Rahul", "Priya", "Amit", "Sneha"],
    "Marks": [85, 90, 75, 95],
    "City": ["Pune", "Mumbai", "Pune", "Nanded"]
})
```

A. Display data

```
print(df)
```

Display the first few rows:

```
print(df.head())
```

Display the last few rows:

```
print(df.tail())
```

You can also specify the number of rows:

```
print(df.head(2))
```

B. Access a column

```
print(df["Name"])
```

To access multiple columns:

```
print(df[["Name", "Marks"]])
```

Remember that a single column selection normally returns a Series, while selecting multiple columns returns a DataFrame.

C. Filter data

Find students who scored more than 80:

```
print(df[df["Marks"] > 80])
```

Find students who belong to Pune:

```
print(df[df["City"] == "Pune"])
```

You can combine conditions using `&` for AND and `|` for OR. Put each condition inside parentheses.

```
print(df[(df["Marks"] > 80) & (df["City"] == "Pune")])
```

D. Add a new column

Suppose you want to add the result of each student.

```
df["Result"] = ["Pass", "Pass", "Pass", "Pass"]
```

```
print(df)
```

Pandas adds the new `Result` column to the DataFrame.

E. Update data

Suppose Rahul's marks change from 85 to 88.

```
df.loc[df["Name"] == "Rahul", "Marks"] = 88

print(df)
```

Here, `.loc` selects rows and columns using labels or Boolean conditions.

F. Delete a column

```
df = df.drop(columns=["City"])

print(df)
```

`drop()` returns a new DataFrame by default. Assigning the result back to `df` keeps the change in your variable.

G. Perform calculations

Find the total marks:

```
print(df["Marks"].sum())
```

Find the average marks:

```
print(df["Marks"].mean())
```

Find the highest marks:

```
print(df["Marks"].max())
```

Find the lowest marks:

```
print(df["Marks"].min())
```

H. Sort data

Sort students by marks in ascending order:

```
print(df.sort_values(by="Marks"))
```

Sort students by marks in descending order:

```
print(df.sort_values(by="Marks", ascending=False))
```

8. Reading CSV and Excel files

Pandas is especially useful when working with external files.

Read a CSV file:

```
import pandas as pd

df = pd.read_csv("students.csv")

print(df)
```

Read an Excel file:

```
df = pd.read_excel("students.xlsx")

print(df)
```

For Excel files, you may need an additional library:

```
pip install openpyxl
```

You can also save a DataFrame to a file.

```
df.to_csv("output.csv", index=False)
```

Or to Excel:

```
df.to_excel("output.xlsx", index=False)
```

The argument `index=False` prevents Pandas from writing the DataFrame index as an additional column.

9. Pandas vs NumPy vs Python lists

Since NumPy and Pandas are both popular Python libraries, it's helpful to understand the difference.

Feature	Python list	NumPy	Pandas
Main purpose	General collections	Numerical computing	Tabular data analysis
Labels	No built-in labels	Mainly integer indexing	Row and column labels
Table operations	Manual	Arrays and matrices	DataFrames
Missing data handling	Manual	Limited/general-purpose	Built-in tools
Read CSV/Excel	Requires additional code	CSV support, other tools for Excel	Convenient file-reading functions

Pandas often uses NumPy internally for numerical operations, so these libraries complement each other.

10. Practice example

Try this small program to test your understanding.

Question: Which Pandas function is used to create a DataFrame from a Python dictionary?

- `pd.Series()`

- `pd.DataFrame()`
- `pd.read_csv()`
- `pd.mean()`

11. What should you learn next?

For a complete Pandas course, I recommend teaching these topics in order:

1. Introduction to Pandas and installation
2. Series and DataFrame
3. Creating DataFrames
4. Reading CSV and Excel files
5. Inspecting data with `head()`, `tail()`, `info()`, and `describe()`
6. Selecting rows and columns using `loc` and `iloc`
7. Filtering and sorting data
8. Adding, updating, and deleting columns
9. Handling missing values with `isna()`, `fillna()`, and `dropna()`
10. Removing duplicates and correcting data types
11. Grouping data using `groupby()`
12. Combining DataFrames using `merge()` and `concat()`
13. Exporting data
14. A real-world data analysis project

Final takeaway: Pandas helps you work with real-world data efficiently. If NumPy is particularly useful for numerical arrays, Pandas is especially useful when your data has rows, columns, and labels—like spreadsheets and database tables.